

EXPERTS LIVE GERMANY 2026

PowerShell Workshop

From First Commands to Daily Productivity

Haiko Hertes

2026-03-04





- Seit 2019 bei SoftwareOne
- Principal Consultant & Architect
- Azure Consulting Team
- Microsoft MVP, YouTuber, Blogger, Conference Speaker
- Familienvater, Offizier d.R., Holzwurm



www.hertes.net



about.me/haiko.hertes



twitter.com/HHertes



youtube.com/c/HaikoHertes

Haiko Hertes

Cloud Architect / Principal Consultant



Agenda #1

- Introduction & Foundations
- Help System, Cmdlets & First Mental Model
- Objects Instead of Text
- Understanding the Pipeline
- Filtering, Selecting, Processing Data

Agenda #2

- Variables, Arrays & Hash Tables
- Loops & Iteration
- Errors, Warnings & Reliable Scripts
- Remoting, Credentials & Administration

THE PROBLEMS WE WANT TO SOLVE

Why this workshop exists

Common daily IT problems

- Repetitive manual tasks
- Clicking through GUIs for simple queries
- Hard to reproduce changes
- Hard to document work
- Different tools for each platform

Goal: Work faster, safer and reproducible





01

Introduction & Foundations

What PowerShell is and why you should care

What PowerShell Actually Is

- PowerShell is:
 - A command-line shell
 - A scripting language
 - An automation framework
 - A management tool for Windows, Server, M365 and Azure
- One interface to control many systems!

THE KEY DIFFERENCE IS THE DATA MODEL

PowerShell vs Traditional Shells

Traditional Shell	PowerShell
Text output	Objects
Parsing required	Structured data
Fragile scripts	Reliable automation
Tools don't integrate	Everything integrates

Key idea: PowerShell handles & passes objects, not text!

PowerShell is available for all major operating systems!

Windows PowerShell vs PowerShell

Windows PowerShell	PowerShell (Core / 7+)
Version 5.1	Version 7+
Windows only	Cross-platform (Windows, Linux, macOS)
Built on .NET Framework	Built on .NET (modern)
Legacy modules guaranteed	Better performance & future features
Still required for some old modules	Recommended for daily work

Important takeaway:

They look identical, but behave differently in modules and compatibility.

How Commands Are Structured

- PowerShell uses a consistent naming scheme:

Verb — **Noun**

- Examples:
 - Get-Service
 - Start-Service
 - Stop-Process
 - Get-ADUser
- Benefits:
 - Discoverable
 - Predictable
 - Learn once, reuse everywhere

YOU ARE NOT EXPECTED TO MEMORIZE COMMANDS

Discoverability

- PowerShell is designed to be explored:
 - Find commands
 - Read built-in help
 - Experiment safely
- (We will do this next)

SHOWING CONSISTENCY AND REMOVING FEAR

DEMO: First Contact with PowerShell

- What we will see:
 - Using first commands
 - Getting current PowerShell Version
 - Some facts about case and whitespace

CONFIRM YOUR ENVIRONMENT WORKS

LAB: First Interaction

- Either use your own computers PowerShell or connect to the Lab VM using SSH:
- `ssh lab\USERNAME@lab0x.lab.local`
- `C:\LabFiles\SetMyPwd.ps1`
- Try out these tasks:

Find out, what PowerShell version you are using

See if there is some Cmdlets to deal with Windows Services

Try out auto-completion for any of the cmdlets

OPT: Switch to PowerShell 7 and repeat first task



Lab Details

WiFi: labLAND-v2
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL
VM: LAB01.lab.local or
LAB02.lab.local
(See sheet)

Username: See sheet

Password: See sheet

SAMPLE: First Interaction

\$PSVersionTable

Get-Command -Noun Service

02

Help, Cmdlets & 1st Mental Model

Google is optional — PowerShell already knows the answer

The Real Skill in PowerShell

You do not learn PowerShell by memorizing commands.

You learn PowerShell by knowing how to:

- search
- inspect
- understand output

The Help system is your primary documentation!

For better readability, you could use the Browser / Online help

Cmdlets Follow Rules

PowerShell commands follow a pattern:

Verb-Noun

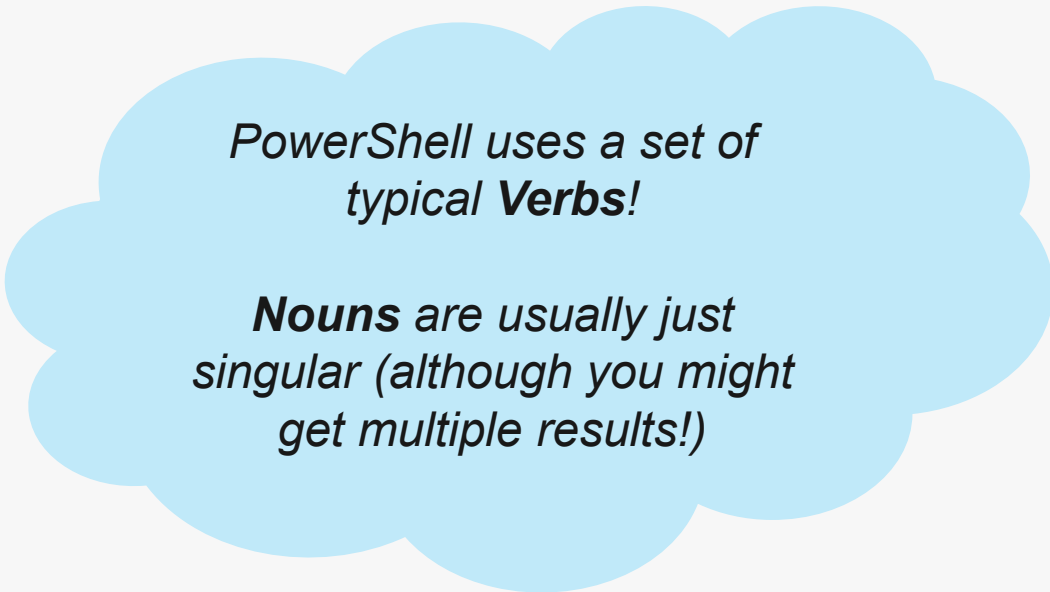
Examples:

- Get-Service
- Get-Process
- Get-EventLog
- Restart-Computer

Meaning:

- Verb = action
- Noun = target

This makes commands “guessable”!



*PowerShell uses a set of
typical **Verbs**!*

***Nouns** are usually just
singular (although you might
get multiple results!)*

[Approved Verbs for PowerShell Commands - PowerShell | Microsoft Learn](#)

START WITH VERBS AND NOUNS YOU ALREADY KNOW

Finding Commands

Use Get-Command to search:

- `Get-Command *service*`
- `Get-Command -Verb Get`
- `Get-Command -Noun Process`

*You rarely start with exact names -
you search first!*



Understanding a Command

Use built-in help:

- `Get-Help Get-Service`
- `Get-Help Get-Service -Examples`
- `Get-Help Get-Service -Full`

Key idea:

- *If you don't know how a command works, ask the command.*



Parameters vs Arguments

Example:

Get-Service -Name Spooler

-Name → parameter
Spooler → argument

*Parameters modify behavior
without changing the command!*



Updating Help

Update-Help

Why:

- Servers often have no internet
- Help is downloaded once
- After that it works offline

(You may need admin rights)



From Command to Understanding Output

We now know:

- how to find commands
- how to read documentation

Next step:

- understanding what commands actually return
- (Leads into object chapter)

PS C:\> Get-UserInfo

Name	Username	Role	Department	Status	Last Login
Alice Johnson	ajohnson	Administrator	IT	Active	2022-12-15
Brian Smith	bsmith	Manager	Sales	Active	2022-11-30
Carol Lee	clee	HR Specialist	Human Resources	Active	2022-12-05
David Brown	dbrown	Developer	Engineering	Inactive	2021-10-20
Emily Davis	edavis	Analyst	Finance	Active	2022-12-10
Frank Miller	fmiller	Support Tech	Support	Active	2022-12-03
Grace Wilson	gwilson	Intern	Marketing	Active	2022-11-28

DEMO: Discover a Command

"We want to find something about services but don't know the command."

"First, I'll list all commands related to services, then I'll check the help examples, and finally, I'll display the running services."

```
PS C:\> Get-Command *service*
```

CommandType	Name	Version	Source
Cmdlet	Get-Service	7.1.3	Microsoft.PowerShell.Management
Cmdlet	New-Service	7.1.3	Microsoft.PowerShell.Management
Cmdlet	Restart-Service	7.1.3	Microsoft.PowerShell.Management
Cmdlet	Set-Service	7.1.3	Microsoft.PowerShell.Management
Cmdlet	Stop-Service	7.1.3	Microsoft.PowerShell.Management

```
PS C:\> Get-Help Get-Service -Examples
```

NAME -----

Get-Service

Example 1: List all services

```
Get-Service
```

Example 2: Get a specific service

```
Get-Service -Name "wuauserv"
```

```
PS C:\> Get-Service
```

Status	Name	DisplayName
Running	wuauserv	Windows Update
Running	bits	Background Intelligent Transfer Service
Stopped	spooler	Print Spooler
Running	WinRM	Windows Remote Management

LAB: Find and Understand a Command

- Either use your own computers PowerShell or connect to the Lab VM using SSH:
- `ssh lab\USERNAME@lab0x.lab.local`
- Try out these tasks:

Search for a cmdlet to list all processes

Use help to get examples for the cmdlet

Run any of the examples given

OPT: Find a command that stops a process and read its help page, but do NOT run it.



Lab Details

WiFi: labLAND-v2
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL
VM: LAB01.lab.local or
LAB02.lab.local
(See sheet)

Username: See sheet

Password: See sheet

THERE IS MORE THAN ONE WAY TO HANDLE THEM

Named vs. Positional Parameters

- `Get-Service -Name „DHCP“`
- `Get-Service „DHCP“`

Syntax Kopieren

```
Get-Service  
  [[-Name] <String[]>  
  [-DependentServices]  
  [-RequiredServices]  
  [-Include <String[]>  
  [-Exclude <String[]>  
  [<CommonParameters>
```

THERE IS MORE THAN ONE WAY TO HANDLE THEM

Named vs. Positional Parameters

- `Get-Service -Name „DHCP“`
- `Get-Service „DHCP“`

**First String
will be used
for `-Name`!**

-Name

Specifies the service names of services to be retrieved. Wildcards are permitted.

Parameter properties

[Expand table](#)

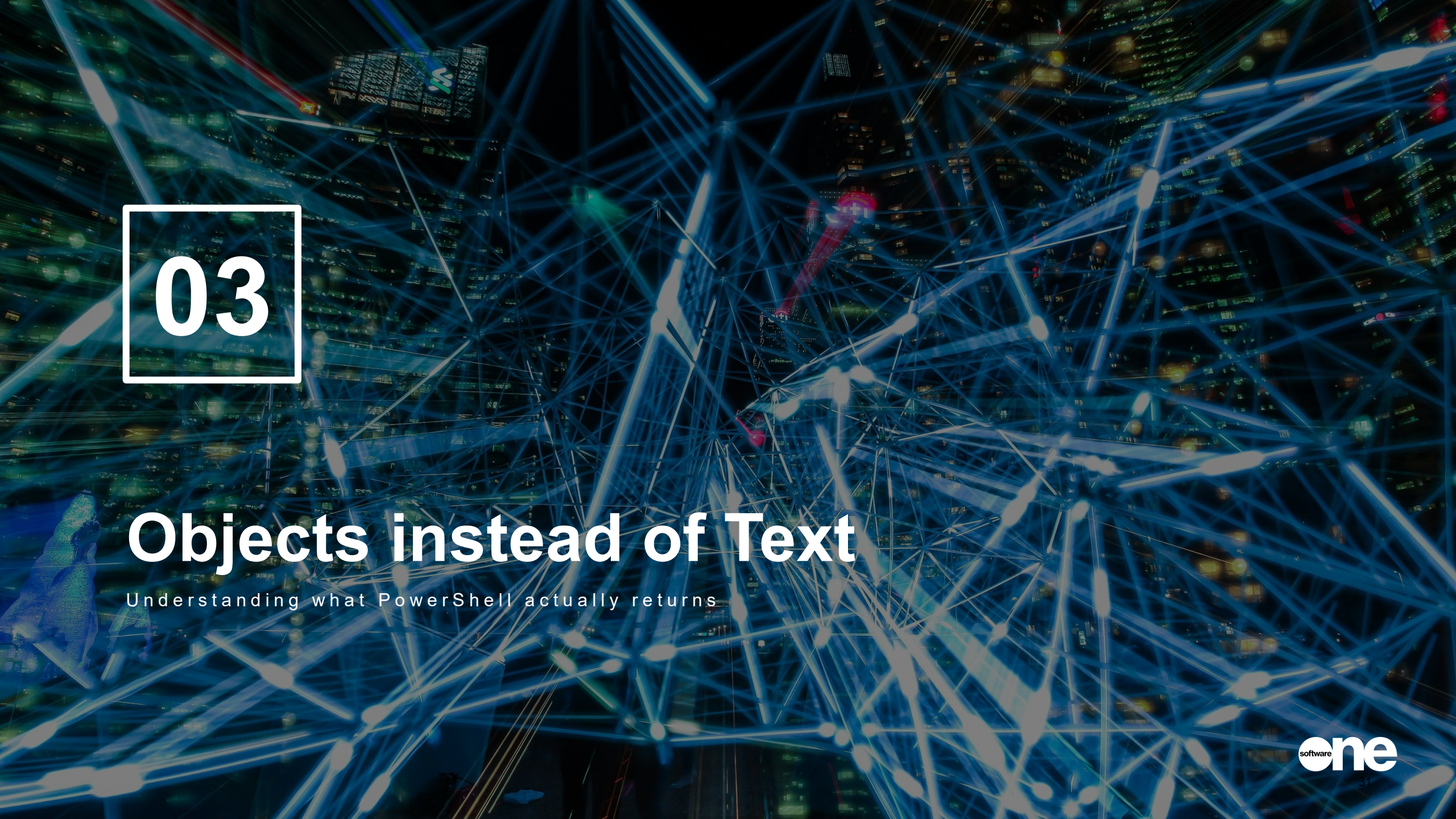
Type:	String[]
Default value:	None
Supports wildcards:	True
DontShow:	False
Aliases:	ServiceName

Parameter sets

Default ^

[Expand table](#)

Position:	0
Mandatory:	False



03

Objects instead of Text

Understanding what PowerShell actually returns

EVERYTHING YOU SEE IS AN OBJECT

The Most Important Concept in PowerShell

In traditional shells:

- commands return text

In PowerShell:

- commands return objects

*You are not reading output —
you are looking at a formatted representation
of data.*



What Is an Object?

An object consists of:

- **Type** — what kind of thing it is
- **Properties** — information about it
- **Methods** — actions it can perform

Example:

*A service is not a line of text —
it is a structured data record.*



THE ATTRIBUTES YOU CAN FILTER ON

Properties

Example service properties:

- Name
- Status
- DisplayName
- StartType

*You do not parse text —
you select properties.*



Methods

Objects are not passive data.
They (can) contain actions:

Examples:

- `.Start()`
- `.Stop()`
- `.Kill()`

PowerShell cmdlets usually call these methods for you.



ONE COMMAND CAN FEED ANOTHER

Passing Results to Another Command

PowerShell commands can send their results to the next command:

- `Command1 | Command2`

Meaning:

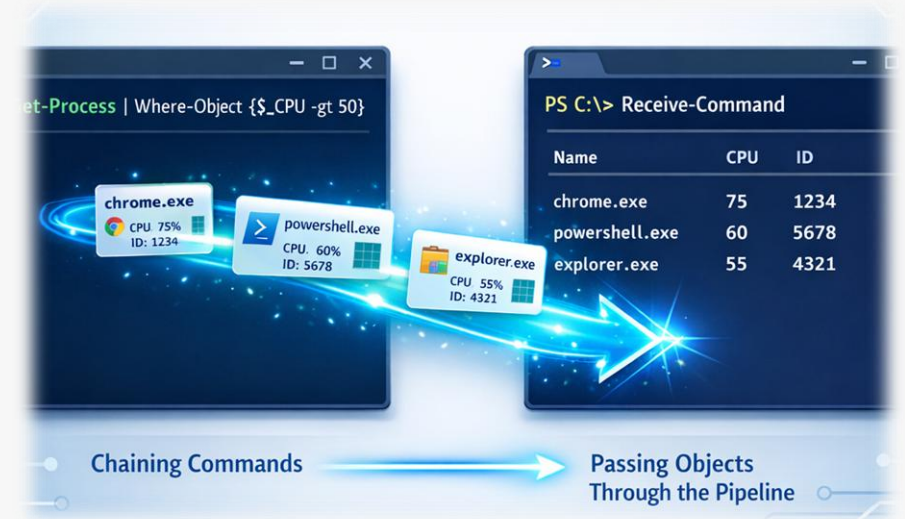
- *“Take the output of the first command and give it to the next one”*

Example:

- `Get-Process | Sort-Object CPU`
- `Get-Process | Sort-Object CPU -Descending | Select-Object -First 5`

We are not combining text — we are passing objects.

- (Detailed pipeline behavior comes next chapter)



Inspecting Objects

Use:

- `Get-Service | Get-Member`

This reveals:

- object type
- properties
- methods

This is the most important diagnostic command in PowerShell.

Why This Matters

Because we work with properties:

- `Get-Service | Where-Object Status -eq Running`

BAD (text parsing mindset)

- searching strings
- depending on formatting
- breaking after updates

YOU NOW KNOW HOW TO INSPECT ANY COMMAND

From Seeing to Understanding

You can now:

- discover properties
- understand outputs
- prepare for filtering

Next step:

- using these properties in the pipeline

REVEAL THE STRUCTURE BEHIND THE OUTPUT

DEMO: Explore an Object live

- *What we see is not (only) what we get...*

LAB: Inspect Objects Yourself

- Either use your own computers PowerShell or connect to the Lab VM using SSH:
- `ssh lab\USERNAME@lab0x.lab.local`
- Try out these tasks:

Run a command that lists all current processes

Inspect its output object structure

Find at least one useful property

OPT: Find a property that shows memory usage and display only that property for all processes.



Lab Details

WiFi: labLAND-v2
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL
VM: LAB01.lab.local or
LAB02.lab.local
(See sheet)

Username: See sheet
Password: See sheet

A person in a white lab coat is looking through a red microscope. The background is a soft, out-of-focus laboratory setting. A large white square with the number '04' is overlaid on the left side of the image.

04

Understanding the Pipeline

The assembly line for your commands

What the Pipeline Really Is

The pipe operator **|**
means:

“Send the result of one command into another command”

Not text

Not copy & paste

Actual structured data! (objects)

EACH COMMAND PERFORMS ONE TASK

Think of It as a Workflow

Example question:

“Which processes currently use the most CPU?”

Get-Process | Sort-Object CPU -Descending | Select-Object -First 5

Steps:

1. Collect data
2. Organize it
3. Reduce it

Pipeline vs Variables

Variable approach:

```
$procs = Get-Process  
$procs | Sort-Object CPU
```

Pipeline approach:

```
Get-Process | Sort-Object CPU
```

Pipeline advantages:

- Faster
- Cleaner
- Less memory
- Natural workflow

But:

- Using variables is re-usable
- Variable approach might improve readability!

But look at this:



- `Get-Process „MsMpEng“ | Stop-Process`



- `Get-Service „W32Time“ | Restart-Service`



- `Get-Service sshd | Stop-Process -WhatIf`



```
PS C:\Users\haiko> Get-Process MsMpEng | Stop-Process -WhatIf
What if: Performing the operation "Stop-Process" on target "MsMpEng (2620)".
PS C:\Users\haiko> Get-Service W32Time | Restart-Service -WhatIf
What if: Performing the operation "Restart-Service" on target "Windows Time (W32Time)".
PS C:\Users\haiko> Get-Service sshd | Stop-Process -WhatIf
What if: Performing the operation "Stop-Process" on target "sshd (2628)".
What if: Performing the operation "Stop-Process" on target "sshd (3268)".
What if: Performing the operation "Stop-Process" on target "sshd (3580)".
```

- Get-Process „MsMpEng“ | Stop-Process



- Get-Service „W32Time“ | Restart-Service



- Get-Service sshd | Stop-Process -WhatIf



Wait - why does this behave differently?



- `Get-Service „W32Time“ | Restart-Service`



- `Get-Service sshd | Stop-Process -WhatIf`



•



- `Get-Item ".\test.txt" | Rename-Item "newname.txt" -WhatIf`



```

PS C:\Users\haiko> Get-Item ".\test.txt" | Rename-Item "newname.txt" -WhatIf
Rename-Item : The input object cannot be bound to any parameters for the command
either because the command does not take pipeline input or the input and its
properties do not match any of the parameters that take pipeline input.
At line:1 char:25
+ Get-Item ".\test.txt" | Rename-Item "newname.txt" -WhatIf
+
+ CategoryInfo          : InvalidArgument: (C:\Users\haiko\test.txt:PSObject) [Re
name-Item], ParameterBindingException
+ FullyQualifiedErrorId : InputObjectNotBound,Microsoft.PowerShell.Commands.Renam
eItemCommand

PS C:\Users\haiko>

```

- `Get-Service sshd | Stop-Process -WhatIf`



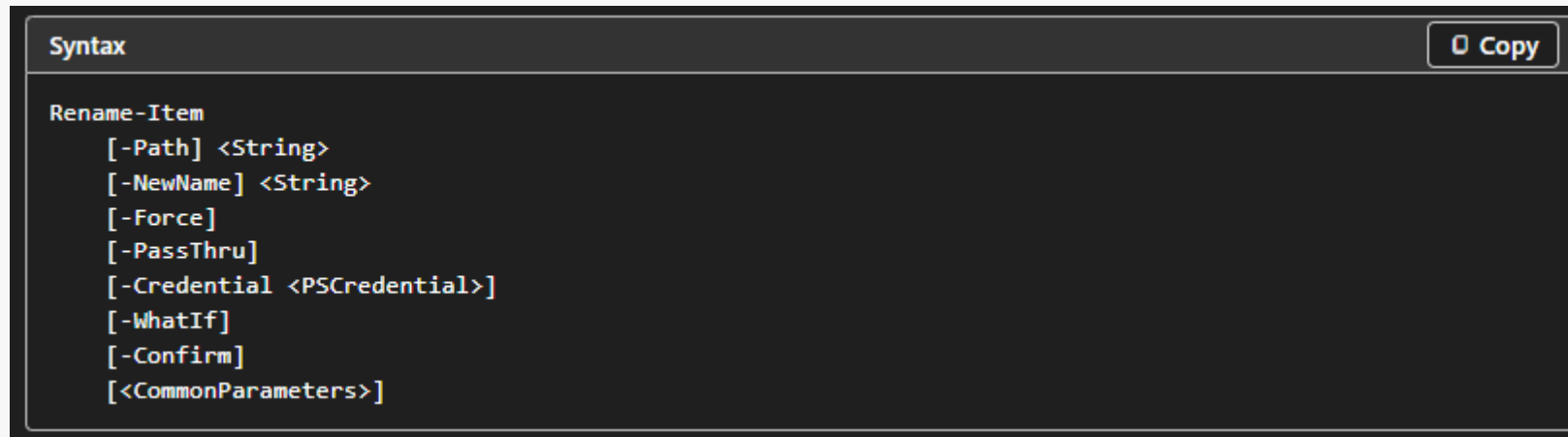
- `Get-Item ".\test.txt" | Rename-Item "newname.txt" -WhatIf`



Please Get-Help!

`Get-Item ".\test.txt" | Rename-Item "newname.txt"`

- `Get-Help Rename-Item -Online`



The screenshot shows a dark-themed window titled 'Syntax' with a 'Copy' button in the top right corner. The content lists the parameters for the 'Rename-Item' command:

```
Rename-Item
  [-Path] <String>
  [-NewName] <String>
  [-Force]
  [-PassThru]
  [-Credential <PSCredential>]
  [-WhatIf]
  [-Confirm]
  [<CommonParameters>]
```

- 2 Parameters are mandatory: Path and NewName
 - We have provided one of them „positional“ (no param name, just the value)
 - The rest could be provided from the Pipeline

Please Get-Help!

- Get-Help Rename-Item -Online

Problem:

There is a given order for handling parameters!

- 1) Named
- 2) Positional
- 3) Pipeline

So „newname.txt“ is used for -Path

```
Get-Item ".\test.txt" | Rename-Item "newname.txt"
```

-Path

Specifies the path of the item to rename.

Parameter properties

[Expand table](#)

Type:	String
Default value:	None
Supports wildcards:	False
DontShow:	False

Parameter sets

ByPath ^

[Expand table](#)

Position:	0
Mandatory:	True
Value from pipeline:	True
Value from pipeline by property name:	True
Value from remaining arguments:	False

Please Get-Help!

```
Get-Item ".\test.txt" | Rename-Item "newname.txt"
```

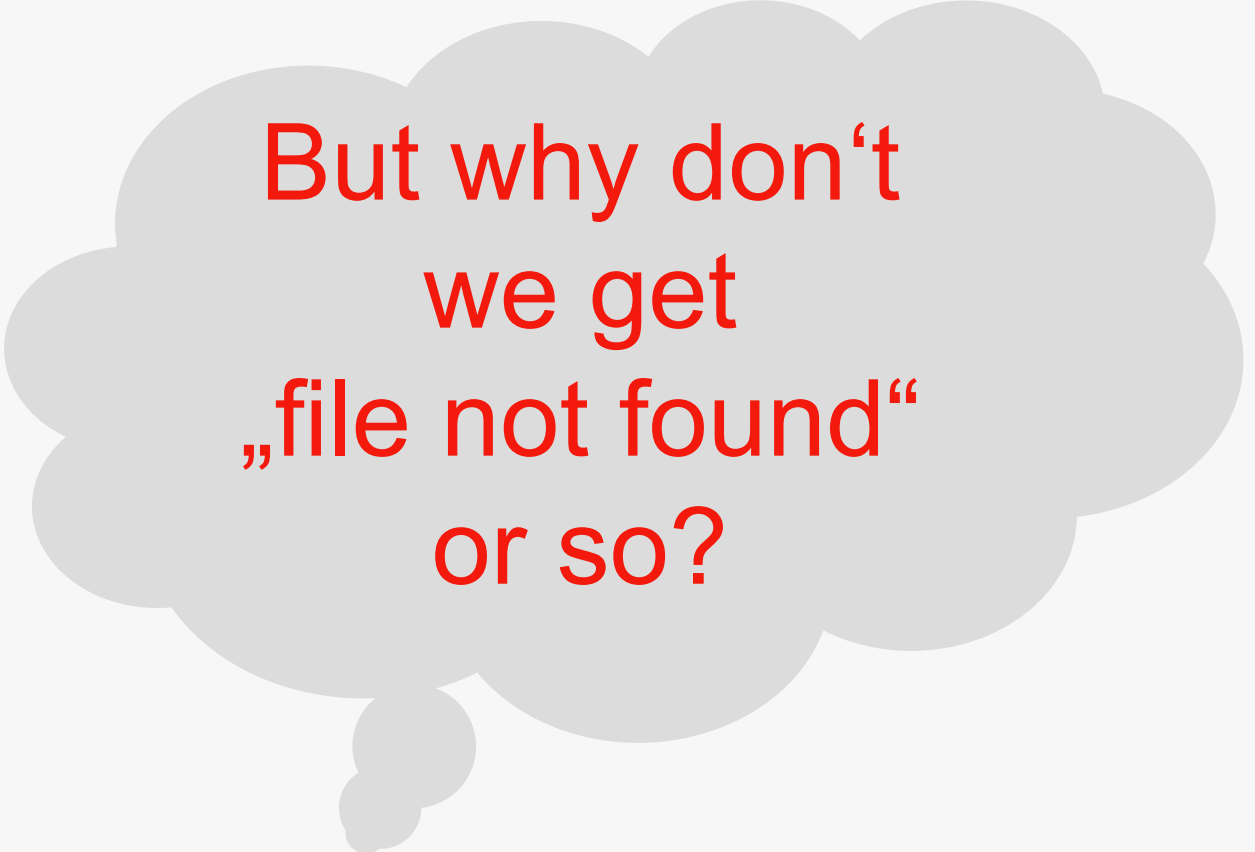
- Get-Help Rename-Item -Online

Problem:

There is a given order for handling parameters!

- 1) Named
- 2) Positional
- 3) Pipeline

So „newname.txt“ is used for -Path



But why don't
we get
„file not found“
or so?

Please Get-Help!

- Get-Help Rename-Item -Online

Problem:

There is a given order for handling parameters!

- 1) Named
- 2) Positional
- 3) Pipeline

The 2nd mandatory parameter does not allow values from pipeline!

Get-Item ".\test.txt" | Rename-Item "newname.txt"

-NewName

Specifies the new name of the item. Enter only a name, not a path and name. If you enter a path that differs from the path that is specified in the **Path** parameter, **Rename-Item** generates an error. To rename and move an item, use **Move-Item**.

You can't use wildcard characters in the value of the **NewName** parameter. To specify a name for multiple files, use the **-replace** operator in a regular expression. For more information about the **-replace** operator, see [about_Comparison_Operators](#).

Parameter properties

 Expand table

Type:	String
Default value:	None
Supports wildcards:	False
DontShow:	False

Parameter sets

(All) ^

 Expand table

Position:	1
Mandatory:	True
Value from pipeline:	False

How to fix this?

- Telling PowerShell what to use the given information for!

```
Get-Item ".\test.txt" | Rename-Item -NewName "newname.txt" -WhatIf
```

```
PS C:\Users\haiko> Get-Item ".\test.txt" | Rename-Item -NewName "newname.txt" -WhatIf  
What if: Performing the operation "Rename File" on target "Item: C:\Users\haiko\test.tx  
t Destination: C:\Users\haiko\newname.txt".
```

Now „newname.txt“ is used for -NewName and
the remaining Parameter -Path is satisfied by the pipeline input!

How to fix this?

- Telling PowerShell what to use the given

Get-Item ".\test.txt" | Rename-Item

```
PS C:\Users\haiko> Get-Item ".\test.txt" | Rename-Item
What if: Performing the operation "Rename File" on destination "C:\Users\haiko\newname.txt".
```

Now „newname.txt“ is used for -NewName and the remaining Parameter -Path is satisfied

-Path

Specifies the path of the item to rename.

Parameter properties

[Expand table](#)

Type:	String
Default value:	None
Supports wildcards:	False
DontShow:	False

Parameter sets

ByPath ^

[Expand table](#)

Position:	0
Mandatory:	True
Value from pipeline:	True
Value from pipeline by property name:	True
Value from remaining arguments:	False

So How Does PowerShell Decide?

PowerShell decides in this order:

- Named parameters
- Positional parameters
- Pipeline input

If no match exists → Error!

DATA CAN BE PASSED DIRECTLY OR THROUGH THE PIPELINE

The -InputObject Parameter

Many PowerShell commands accept an object explicitly:

```
$svc = Get-Service [-Name] Spooler  
Stop-Service -InputObject $svc
```

The same action using the pipeline:

```
Get-Service Spooler | Stop-Service
```

Both do the same thing - but the pipeline is shorter and chains naturally.

What Changes When Using the Pipeline

When using a parameter:

- You decide where the data goes
- You must store it first

When using the pipeline:

- PowerShell automatically places the object
- Commands connect naturally

So this:

```
$proc = Get-Process notepad  
Stop-Process -InputObject $proc
```

becomes:

```
Get-Process notepad | Stop-Process
```

Objects Travel Through the Pipeline

PowerShell does not pass formatted output.

It passes:

- objects
- with properties
- with types

This is why filtering, sorting and such works reliably!

REVEAL THE STRUCTURE BEHIND THE OUTPUT

DEMO: Building and Breaking the Pipeline

- Sorting
- Filtering
- Reducing
- Selecting

REVEAL THE STRUCTURE BEHIND THE OUTPUT

DEMO: Building and Breaking the Pipeline

- Sorting
- Filtering
- Reducing
- Selecting
- Pipeline Details

REVEAL THE STRUCTURE BEHIND THE OUTPUT

DEMO: Building and Breaking the Pipeline

- Sorting
- Filtering
- Reducing
- Selecting
- Pipeline Details
- InputObject

LAB: Build and Refine a Pipeline

- Either use your own computers PowerShell or connect to the Lab VM using SSH:
- `ssh lab\USERNAME@lab0x.lab.local`
- Try out these tasks:

Find out what process uses the most CPU performance.

Find all running Windows services and sort them by display name.

OPT: Try this out:

*Get-Process | Select-Object Name | Sort-Object CPU
Why does it behave as it does? What needs to be changed?*



Lab Details

WiFi: labLAND-v2
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL
VM: LAB01.lab.local or
LAB02.lab.local
(See sheet)

Username: See sheet
Password: See sheet

Sample solutions

- `Get-Process |
Sort-Object CPU -Descending |
(Select-Object -First 1 Name, CPU)`
- `Get-Service |
Where-Object Status -eq Running |
Sort-Object Name |
Select-Object Name, Status`
- Selecting “drops” all other properties, so you can’t use them anymore!
- `Get-Process |
Sort-Object CPU -Descending |
Select-Object Name`

Let's have a break!
*Please be back and
ready in 10 mins*



A green wooden door with a rusty chain and padlock. The door is made of vertical wooden planks painted green. A heavy, rusty metal chain is draped across the door, secured by a large, dark metal padlock in the center. The chain is attached to two points on the door, one on the left and one on the right. The padlock is a simple, rectangular metal lock. The background is a solid green color, matching the door's paint. The overall image has a slightly grainy, textured appearance.

05

Parameter Binding & Positional Arguments

How PowerShell decides where your data goes

The Binding Order

PowerShell processes parameters in this order:

- Named parameters
- Positional parameters
- Pipeline input

If no valid match exists -> error

EXPLICIT ALWAYS WINS

Named Parameters

```
Rename-Item -Path ".\test.txt" -NewName "newname.txt"
```

You explicitly tell PowerShell:

- What goes to -Path
- What goes to -NewName

No ambiguity...

Positional Parameters

Some parameters are positional.

- `Rename-Item ".\test.txt" "newname.txt"`

Here:

- First value = `-Path` (Pos. 0)
- Second value = `-NewName` (Pos. 1)

PowerShell maps based on declared position.

You can check positions with:

- `Get-Help Rename-Item -Full`

Pipeline Binding

If no named or positional argument is provided, PowerShell attempts pipeline binding.

Two mechanisms exist:

- ByValue
- ByPropertyName

We now examine both!

ByValue Binding

If the parameter type matches the incoming object type, PowerShell binds automatically.

Example:

- `Get-Service sshd | Stop-Service`

`Stop-Service -InputObject` accepts `ServiceController` objects.
So binding happens by type.

`“W32Time” | Get-Service`

-> `“String”` maps to `“String”` and `“-name”` is the first parameter accepting Strings!

ByPropertyName Binding

If a parameter name matches a property name, PowerShell binds using that property.

Example:

- `Get-Service sshd | Stop-Process -WhatIf`

Why it works:

- Service object has property Name
- Stop-Process has parameter -Name
- Binding happens via property name

This is powerful - and dangerous!

Why Rename-Item Failed Earlier

```
Get-Item ".\test.txt" | Rename-Item "newname.txt"
```

What happened:

- "newname.txt" was treated as positional -Path
- That satisfied -Path
- No pipeline binding left for -Path
- -NewName remained unassigned

Fix:

```
Get-Item ".\test.txt" | Rename-Item -NewName "newname.txt"
```

DEMO: Make Binding Visible

- Get-ChildItem uses „byValue“
- Stop-Process uses „byValue“
- Get-Service uses „byPropertyName“
- `[PSCustomObject]@{ Name = 'W32Time' } | Get-Service`
- Error when no match

DEMO: Make Binding Visible

- Get-ChildItem uses „byValue“
- Stop-Process uses „byValue“
- Get-Service uses „byPropertyName“
- [PSCustomObject]@{ Name = 'W32Time' } | Get-Service
- Error when no match

```
PS C:\Users\haiko> Get-Help Get-Service -Parameter Name
-Name <string[]>

        Required?                false
        Position?                0
        Accept pipeline input?    true (ByValue, ByPropertyName)
        Parameter set name        Default
```

PREDICT WHAT POWERSHELL WILL DO BEFORE YOU RUN IT

LAB: Parameter Binding in Practice

Run the command and explain how the arguments are mapped:
`Copy-Item ".\test.txt" ".\backup.txt" -WhatIf`

Run and explain why pipeline input works:
`Get-Service sshd | Select-Object -First 1`
Then verify which object type is passed:
`Get-Service sshd | Get-Member`

Run and explain why this works:
`Get-Service sshd | Select-Object Name`
Now show that the **Name property alone** can be piped as strings:
`Get-Service sshd | Select-Object -ExpandProperty Name`

OPT: Binding Inspection

For each parameter below, determine whether it accepts:
pipeline input **ByValue** or **ByPropertyName**

Inspect:

`Stop-Process -Name`

`Restart-Service -InputObject`



Lab Details

WiFi: labLAND-v2
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL
VM: LAB01.lab.local or
LAB02.lab.local
(See sheet)

Username: See sheet

Password: See sheet

SAMPLES: Parameter Binding in Practice (1/2)

- `Copy-Item ".\test.txt" ".\backup.txt" -WhatIf`

Mapping:

- `".\test.txt" → -Path (Position 0)`
- `".\backup.txt" → -Destination (Position 1)`

- `Get-Service sshd | Get-Member`

- `Type: System.ServiceProcess.ServiceController`

=> Pipeline passes the object, not text.

- `Get-Service sshd | Select-Object -ExpandProperty Name`
- Result becomes a string list (Name values), not service objects.

SAMPLES: Parameter Binding in Practice (2/2)

```
(Get-Command Stop-Process).Parameters["Name"].Attributes
```

Look for:

- ValueFromPipelineByPropertyName

```
(Get-Command Restart-Service).Parameters["InputObject"].Attributes
```

Look for:

- ValueFromPipeline



06

Filtering & Processing

Now we make PowerShell actually useful

TOO MUCH DATA IS USELESS DATA

The problem

Get-Process

Reality:

- Hundreds of lines
- Hard to read
- Not actionable

Goal:

- Filter. Reduce. Extract.

Filtering with Where-Object

Syntax:

- `Command | Where-Object condition`
- `Command | Where-Object { filter script }`

Example:

- `Get-Service | Where-Object Name -eq "DHCP"`
- `Get-Service | Where-Object { $_.Status -eq "Running" }`

Filter Script:

All items are returned, where the script is "true"

REFERRING TO EACH ITEM IN THE PIPELINE

The Current Object (\$_)

Inside a script block:

- \$_ (or \$PSItem)

means:

- The object currently being processed

Example:

- `Get-Process | Where-Object { $_.CPU -gt 100 }`

This checks each item individually!

Processing with ForEach-Object

Filtering removes objects. Processing modifies or outputs something.

Example:

- `Get-Service | ForEach-Object { $_.Name }`

or:

- `Get-Process |
ForEach-Object {
 "$($_.Name) uses $(($_.CPU))"
}`

or:

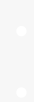
- `Get-ADUser -Filter * -Properties Department |
Where-Object { $_.Department -eq "Finance" } |
ForEach-Object {
 Add-ADGroupMember -Identity "GG_Finance_Department" -Members $_
}`

DEMO: Practical Filtering

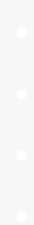
Find stopped services

- 

Find large files

- - 

Find specific users

- - 

DEMO: Practical Filtering

Find stopped services:

- `Get-Service | Where-Object { $_.Status -eq "Stopped" }`

Find large files:

- `Get-ChildItem -File |
Where-Object { $_.Length -gt 1MB }`

Find specific users:

- `Get-ADUser -Filter 'Enabled -eq $false' |
ForEach-Object {
 Enable-ADAccount -Identity $_ -WhatIf
}`

LAB: Filtering & Processing

Display only the running services!

Display processes using more than **100 MB memory**.

For each Service, which is NOT running, display something like
“Service A is not running, and has a startup type of...”

OPT: Combine Conditions

Display processes:

Using more than 50 MB memory

AND with CPU usage greater than 1



Lab Details

WiFi: labLAND-v2
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL
VM: LAB01.lab.local or
LAB02.lab.local
(See sheet)

Username: See sheet

Password: See sheet

SAMPLE: Filtering & Processing

- **Task 1 - Display only the running services**
- `Get-Service |
Where-Object { $_.Status -eq "Running" }`
- **Task 2 - Display processes using more than 100 MB memory**
- `Get-Process | Get-Member
Get-Process |
Where-Object { $_.WorkingSet -gt 100MB }`
- **Task 3 - For each service which is NOT running, display a message like...**
- `Get-Service |
Where-Object { $_.Status -ne "Running" } |
ForEach-Object {
 "Service $($_.Name) is not running, and has a startup type of $($_.StartType)."
}`

SAMPLE: Filtering & Processing

- **OPT Combine Conditions**
- „Display processes using more than 50 MB memory AND with CPU usage greater than 1“
- ```
Get-Process |
Where-Object {
 $_.WorkingSet -gt 50MB -and
 $_.CPU -gt 1
}
```

compact version:

- ```
Get-Process |  
Where-Object { $_.WorkingSet -gt 50MB -and $_.CPU -gt 1 }
```

SAMPLE: Filtering & Processing

- **OPT Combine Conditions**
- „Display processes using more than 50 MB memory AND with CPU usage greater than 1“
- ```
Get-Process |
Where-Object {
 $_.WorkingSet -gt 50MB -and
 $_.CPU -gt 1
}
```

compact version:

- ```
Get-Process |  
Where-Object { $_.WorkingSet -gt 50MB -and $_.CPU -gt 1 }
```

```
Get-Process | Where-Object WorkingSet -gt 50MB  
Get-Process | Where-Object WorkingSet -gt 50MB -and CPU -gt 1  
Get-Process | Where-Object $_.WorkingSet -gt 50MB -and $_.CPU -gt 1  
Get-Process | Where-Object { $_.WorkingSet -gt 50MB -and $_.CPU -gt 1 }
```



07

Loops, Arrays & Practical Automation

Because one server is never enough

When Is a Loop Needed?

So far, we used:

- `Get-Service | ForEach-Object { ... }`

But sometimes we need:

- Reuse data
- Store multiple items
- Iterate multiple times
- Combine logic

That's when we use a loop!

Arrays

An array holds multiple items:

- `$servers = @("Server01", "Server02", "Server03")` or
- `$servers = ("Server01", "Server02", "Server03")` or
- `$servers = "Server01", "Server02", "Server03"`

You can access items by index:

- `$servers[0]`

Arrays are useful when:

- Looping through servers
- Processing multiple paths
- Working with lists

The foreach Loop (Language Keyword)

Syntax:

- ```
foreach ($server in $servers) {
 # do something
}
```

Example:

- ```
$servers = @("Server01","Server02")
```

```
foreach ($server in $servers) {  
    "Checking $server"  
}
```

foreach vs ForEach-Object

foreach (keyword)	ForEach-Object
Works on stored collection	Works in pipeline
Slightly faster	More natural in pipelines
Requires variable	No variable needed

Pipeline:

- `Get-Service | ForEach-Object { $_.Name }`

Language loop:

- ```
$services = Get-Service
foreach ($service in $services) {
 $service.Name
}
```

*Same result - different approach!*

# DEMO: Automating Across Multiple Servers

## Scenario

We want to query services from two lab machines:

- Lab01
- Lab02

Instead of running the command manually twice, we automate it.

# DEMO: Automating Across Multiple Servers

```
$servers = @("Lab01","Lab02")
```

```
foreach ($server in $servers) {
 "==== $server ====
 Invoke-Command -ComputerName $server -ScriptBlock {
 Get-Service | Where-Object { $_.Status -eq "Running" }
 }
}
```

## DEMO: Automating Across Multiple Servers

```
$results = foreach ($server in $servers) {
 Invoke-Command -ComputerName $server -ScriptBlock {
 Get-Service
 } | ForEach-Object {
 [PSCustomObject]@{
 Server = $server
 Service = $_.Name
 Status = $_.Status
 }
 }
}
$results | Export-Csv ".\ServiceReport.csv" -NoTypeInfo
```

# LAB: Looping Through AD Data

We want to:

- Query users from multiple departments
- Display selected information
- Build structured output

Create a Department Array with „IT“ and „Finance“. Use a foreach loop to display the department names.

For each department:

- Retrieve users from that department
- Display Name and Department

*(Hint: use `Get-ADUser -Filter * -Properties ...`)*

**OPT:** Create a new custom object containing:

- Department
- User Name
- Enabled Status

Store everything in a variable and display the result.



## Lab Details

WiFi: labLAND-v2  
(or „-5G“ for 5GHz)

Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL

VM: LAB01.lab.local or  
LAB02.lab.local  
(See sheet)

Username: See sheet

Password: See sheet

## SAMPLE: Looping Through AD Data

### Task 1 – Department Array

```
$departments = @("IT","Finance")
```

```
foreach ($dept in $departments) {
 $dept
}
```

### Task 2 – Get Users

```
$departments = @("IT","Finance")
```

```
foreach ($dept in $departments) {
 "==== $dept ====
 Get-ADUser -Filter "Department -
eq '$dept'" -Properties Department |
 Select-Object Name, Department
}
```

```
$departments = @("IT","Finance")
```

```
foreach ($dept in $departments) {
 "==== $dept ====
 Get-ADUser -Filter * -Properties Department
 |
 Where-Object { $_.Department -eq $dept } |
 Select-Object Name, Department
}
```



## SAMPLE: Looping Through AD Data

### Task 3 - Custom Object Report

```
$departments = @("IT","Finance")
```

```
$report = foreach ($dept in $departments) {
 Get-ADUser -Filter "Department -eq '$dept'" -Properties Department,
Enabled |
 ForEach-Object {
 [PSCustomObject]@{
 Department = $dept
 Name = $_.Name
 Enabled = $_.Enabled
 }
 }
}
```

```
$report
```

08

# Errors & Safe Admin

Because things will fail

Total Deaths

4.720

Total Recovered

68.324

3.056 deaths  
**Hubei** China

827 deaths  
Italy

429 deaths  
Iran

66 deaths  
Korea, South

55 deaths  
Spain

48 deaths  
**France** France

31 deaths  
**Washington** US

22 deaths  
**Henan** China

16 deaths  
Japan

13 deaths  
**Heilongjiang** China

8 deaths

50.318 recovered  
**Hubei** China

2.959 recovered  
Iran

1.289 recovered  
**Guangdong** China

1.249 recovered  
**Henan** China

1.197 recovered  
**Zhejiang** China

1.045 recovered  
Italy

999 recovered  
**Hunan** China

984 recovered  
**Anhui** China

934 recovered  
**Jiangxi** China

734 recovered  
**Shandong** China

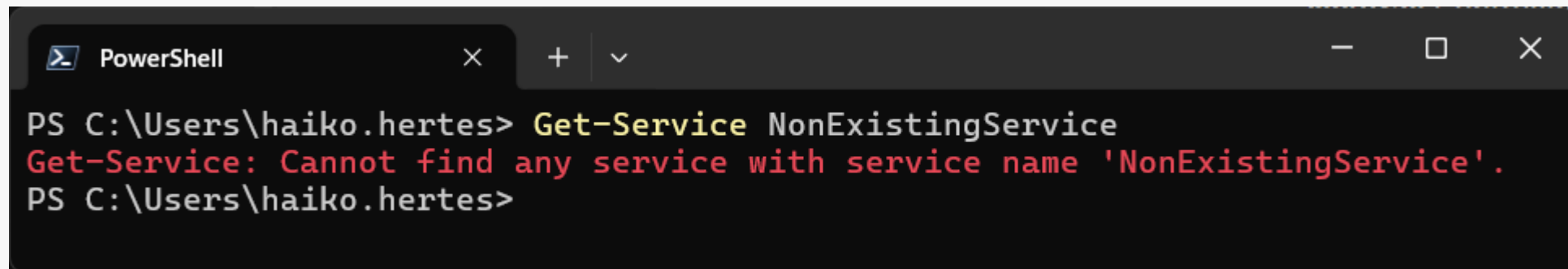
629 recovered  
**Jiangsu** China

# Not All Messages Are (Created) Equal

## PowerShell can produce:

- Output (normal objects)
- Warnings (yellow)
- Errors (red)
- Verbose messages
- Debug messages

## Example:

A screenshot of a PowerShell terminal window. The window has a title bar with 'PowerShell' and standard Windows window controls (minimize, maximize, close). The terminal shows a command prompt 'PS C:\Users\haiko.hertes>' followed by the command 'Get-Service NonExistingService'. The output is an error message: 'Get-Service: Cannot find any service with service name 'NonExistingService''. The error message is displayed in red text. The prompt 'PS C:\Users\haiko.hertes>' is shown again on the next line.

```
PS C:\Users\haiko.hertes> Get-Service NonExistingService
Get-Service: Cannot find any service with service name 'NonExistingService'.
PS C:\Users\haiko.hertes>
```

SHALL WE PROCEED?

## Non-Terminating vs Terminating Errors

- Non-terminating error:  
    >> Execution continues!
- Terminating error:  
    >> Execution stops!

*Not every red message stops your script!*

```
PS C:\Users\haiko.hertes>
Get-Service NonExistingService
Write-Host "Still running"

Throw "An error occurred"
Write-Host "This will not be printed"
Get-Service:
Line |
 2 | Get-Service NonExistingService
 | ~~~~~
 | Cannot find any service with service name 'NonExistingService'.
Still running
Exception:
Line |
 5 | Throw "An error occurred"
 | ~~~~~
 | An error occurred
PS C:\Users\haiko.hertes> S
```

# Controlling Error Behavior

## Example:

```
Get-Service NonExistingService -ErrorAction SilentlyContinue
```

## Common values:

- Continue (default)
- SilentlyContinue
- Stop
- Ignore

## Example forcing terminating behavior:

```
Get-Service NonExistingService -ErrorAction Stop
```

*Important for try/catch!*

## Try / Catch (Basic Pattern)

### Syntax:

```
try {
 # risky command
}
catch {
 # error handling
}
```

### Example:

```
try {
 Get-Service NonExistingService -ErrorAction Stop
}
catch {
 "Service not found."
}
```

SAFE EXECUTION BEFORE CHANGING ANYTHING

## **-WhatIf and -Confirm**

Many administrative cmdlets support:

- WhatIf
- Confirm

### **Example:**

```
Remove-Item .\test.txt -WhatIf
```

*This provides a safety-net!*



# DEMO: Safe Administration in Practice

- We will see:
  - Non-Terminating Error
  - Terminating Error
  - Try/Catch
  - WhatIf
  - Confirm

# DEMO: Safe Administration in Practice

```
Write-Host "Script continues"
Get-Service NonExistingService -ErrorAction Stop
// Terminates here!
```

```
try {
 Get-Service NonExistingService -ErrorAction Stop
}
catch {
 "Handled the error safely."
}
Stop-Service W32Time -WhatIf
Stop-Service W32Time -Confirm
```

# LAB: Safe Execution

Run a command that produces an error.  
Ensure the script continues running.

Force the same command to stop execution using -ErrorAction.

Wrap the command in a try/catch block and display a custom message.

## OPT:

Test a cmdlet that supports -WhatIf and verify behavior.



## Lab Details

WiFi: labLAND-v2  
(or „-5G“ for 5GHz)  
Password: 1a2b3c4d5e6f

Domain: LAB.LOCAL  
VM: LAB01.lab.local or  
LAB02.lab.local  
(See sheet)

Username: See sheet  
Password: See sheet

# SAMPLE: Safe Execution

## Task 1

```
Get-Service NonExistingService
Write-Host "Still running"
```

## Task 2

```
Get-Service NonExistingService -ErrorAction Stop
```

## Task 3

```
try {
 Get-Service NonExistingService -ErrorAction Stop
}
catch {
 "The service does not exist."
}
```

## OPT

```
Remove-Item .\test.txt -WhatIf
```

# Best Practices & Sustainable Usage

Future you will thank you

# Use the Pipeline Properly

## Good:

```
Get-Service |
Where-Object Status -eq Running |
Select-Object Name, Status
```

## Bad:

```
Get-Service |
Format-Table |
Where-Object { $_.Status -eq "Running" }
```

## Rule:

*Never format in the middle of a pipeline.*

*Formatting is for the very end - or not at all in scripts!*

# Prefer Server-Side Filtering

## Instead of:

```
Get-ADUser -Filter * -Properties Department |
Where-Object { $_.Department -eq "IT" }
```

## Prefer:

```
Get-ADUser -Filter "Department -eq 'IT'"
```

## Benefits:

- Faster
- Less network traffic
- Cleaner scripts



## Write Readable Code

### Avoid:

gsv | ? Status -eq Running | % Name  
(Use of Alias in general!)

### Prefer:

Get-Service |  
Where-Object Status -eq Running |  
Select-Object Name

*Abbreviations are fine interactively - but not in scripts.*

# Avoid Hardcoding

## Instead of:

```
Invoke-Command -ComputerName DC01.lab.local ...
```

## Use:

```
param (
 [string]$ServerName
)
```

## Or at least:

```
$servers = @"LAB01","LAB02"@
```

*Make scripts adaptable!*

RED TEXT IS NOT CONTROL FLOW

# Handle Errors Intentionally

## Bad:

```
Get-Service Foo
```

## Better:

```
try {
 Get-Service Foo -ErrorAction Stop
}
catch {
 Write-Warning "Service not found"
}
```

*Intentional behavior > accidental behavior!*

SCRIPTS ARE CODE

## Use Version Control

- Store scripts in Git
- Use comments
- Document assumptions
- Track changes

*Even small automation grows over time!*

09

## Remoting & Wrap-up

# Why Remoting Matters

So far, we worked locally.

**In real environments, you manage:**

- Multiple servers
- Domain controllers
- Azure resources
- Remote services

**PowerShell Remoting allows you to:**

- Execute commands remotely
- Reuse the same logic
- Automate at scale

## Credentials

Create a credential object:

```
$cred = Get-Credential
```

This returns a:

```
System.Management.Automation.PSCredential
```

Use it with remote commands:

```
Invoke-Command -ComputerName Lab01 -Credential $cred -ScriptBlock
{
 Get-Service
}
```



# DEMO: Remoting

- We will see:
  - Using Invoke-Command
  - Using Credentials
  - Using Enter-PSSession
  - Combining few other things into one

## DEMO: Remoting

```
$cred = Get-Credential
Invoke-Command -ComputerName DC01.lab.local -Credential $cred -
ScriptBlock {
 hostname
 Get-WindowsFeature | Where-Object {$_.Installed -eq $true} |
Select-Object -First 5
} | Select Name,DisplayName
```

```
Enter-PSSession -ComputerName LAB01.lab.local -Credential $cred
```

## DEMO: Remoting

```
Invoke-Command -ComputerName DC01.lab.local -ScriptBlock {
 Get-ADUser -Filter * -ResultSetSize 5
}
```

```
Invoke-Command -ComputerName "Lab01.lab.local","Lab02.lab.local"
 -Credential $cred -ScriptBlock {
 [PSCustomObject]@{
 Computer = hostname
 RunningServices = (Get-Service | Where-Object Status -eq
"Running").Count
 }
 }
```

# What you've learned today

- Objects, not text
- Pipelines
- Binding mechanics
- Filtering & processing
- Loops & arrays
- Safe execution
- Remoting

***PowerShell is not a tool - it's a framework for automation.***



Vielen Dank an unsere Sponsoren!

## Diamond



## Platinum



## Gold



## Silber







**Thank you for  
your  
participation!**